

Кременчуцький національний університет імені Михайла Остроградського
(повне найменування вищого навчального закладу)

Кафедра комп'ютерної інженерії та електроніки
(повна назва кафедри)

ЛАБОРАТОРНА РОБОТА № 5

Тема: Обробка текстових даних, рядкові методи та регулярні вирази.

Мета роботи: Опанувати обробку, валідацію та аналіз тексту мовою Python.

Студента III курсу KI-24-1 групи

Ступінь вищої освіти бакалавр
(бакалавр, магістр)

Спеціальність 123 – «Комп'ютерна інженерія»

Освітня програма Програмування на Python. Базовий рівень

Огоновського Олександра Євгенійовича

м. Кременчук 2026 рік

1.1 Порядок виконання роботи

1.1.1 Постановка індивідуального завдання

Варіант 15 передбачає розроблення аналізатора текстового журналу автомобільної шини CAN-bus. Кожний рядок має містити часову мітку в мілісекундах, трисимвольний CAN ID, довжину кадру DLC, масив шістнадцяткових байтів Data_Bytes та стан шини. Перші два байти коректного кадру інтерпретуються як беззнакове ціле у порядку big-endian і задають частоту обертання двигуна за формулою $RPM = raw_value \times 0.25$.

Для розв'язання створено модуль text_sanitizer.py для очищення пробілів і керувальних символів та модуль log_parser.py. Він застосовує скомпільований регулярний вираз з іменованими групами, виконує семантичну валідацію та формує підсумкову таблицю засобами f-рядків.

Таблиця 1.1 – Поля журналу CAN-bus і правила обробки

Поле	Формат у журналі	Перевірка та використання
Timestamp_MS	десятькове ціле	Зберігається як час надходження кадру.
CAN_ID	три hex-символи	Значення має належати 000..7FF.
DLC	ціле число	Допустимий діапазон 1..8; визначає число байтів.
Data_Bytes	hex-байти через пробіл	Кількість байтів повинна збігатися з DLC; два перші байти декодують RPM.
Bus_State	ACTIVE, PASSIVE або OFF	Перевіряється належність до дозволеної множини.

1.1.2 Проєктування регулярного виразу

Шаблон CAN_FRAME_PATTERN прив'язаний до початку та кінця рядка символами ^ і \$. Послідовність \s* допускає зайві пробіли біля роздільника |, а групи (?P<name>...) створюють словник полів через match.groupdict(). Для CAN_ID та Data_Bytes використано клас [0-9A-Fa-f], квантифікатор {3} контролює довжину ідентифікатора, а (?:\s+...)* описує решту масиву байтів.

Функція sanitize_log_line нормалізує рядок до застосування регулярного виразу. Після успішного зіставлення build_frame перетворює текстові групи на числа, перевіряє діапазони

CAN ID і DLC, кількість байтів та стан шини. Некоректні рядки не потрапляють у статистику коректно розібраних кадрів.

1.2 Результати виконання

Під час запуску оброблено вісім рядків даних. Чотири кадри пройшли валідацію, а чотири відхилено: для DLC=9, CAN ID=8A0, невідповідної кількості байтів і порушеного формату рядка. Для коректних кадрів одержано мінімальне значення 0.00 RPM, максимальне 2048.00 RPM та середнє 1494.00 RPM.

1.2.1 Вхідний журнал і консольний звіт

```
File Edit Selection View Go Run Terminal Help
B5 [Administrator]

input_log.txt
1 # Raw, unsplit log: Timestamp Data_Bytes RPM = raw_value * 0.25.
2 # Рядок 4 неможливо відрізнити садию модифікацію в режимі авто-моніторингу.
3 Timestamp_MS: 1800 | CAN_ID: 8C9 | DLC: 8 | Data_Bytes: 1E 20 00 00 00 00 00 | Bus_State: ACTIVE
4 Timestamp_MS: 1250 | CAN_ID: 8C9 | DLC: 9 | Data_Bytes: 1F 40 00 00 00 00 00 00 | Bus_State: ACTIVE
5 Timestamp_MS: 1500 | CAN_ID: 8C9 | DLC: 8 | Data_Bytes: 20 00 00 00 00 00 00 00 | Bus_State: PASSIVE
6 Timestamp_MS: 1750 | CAN_ID: 8C9 | DLC: 8 | Data_Bytes: 00 00 00 00 00 00 00 00 | Bus_State: ACTIVE
7
8 # Наступні рядки мають невірні помилки, щоб показати валідацію.
9 Timestamp_MS: 2000 | CAN_ID: 8C9 | DLC: 9 | Data_Bytes: 1E 20 00 00 00 00 00 00 | Bus_State: ACTIVE
10 Timestamp_MS: 2250 | CAN_ID: 8A0 | DLC: 8 | Data_Bytes: 1E 20 00 00 00 00 00 00 | Bus_State: ACTIVE
11 Timestamp_MS: 2500 | CAN_ID: 8C9 | DLC: 4 | Data_Bytes: 1E 20 00 | Bus_State: ACTIVE
12 Timestamp_MS: 2750 | ID=8C9 | DLC=8 | DATA=1E 20 00 00 00 00 00 00 | STATE=ACTIVE
13

Problems Output Debug Console Terminal Ports
(Power) PS C:\Users\user\Desktop\python\python-labs\lab5> code activate ce_lab_env
(ce_lab_env) PS C:\Users\user\Desktop\python\python-labs\lab5> & C:\ProgramData\miniconda3\envs\ce_lab_env\python.exe c:\Users\user\Desktop\python\python-labs\lab5\log_parser.py
(Power) PS C:\Users\user\Desktop\python\python-labs\lab5>
ПОВІДОМЛЕННЯ: рядок 9: DLC має бути в діапазоні 1-8.
ПОВІДОМЛЕННЯ: рядок 10: CAN ID має належати стандартному 11-бітному діапазону 000-7FF.
ПОВІДОМЛЕННЯ: рядок 11: DLC=4, але в полі Data_Bytes наведено 3 байти(а).
ПОВІДОМЛЕННЯ: рядок 12: Timestamp=2750 | ID=8C9 | DLC=8 | DATA=1E 20 00 00 00 00 00 00 | STATE=ACTIVE

=====
RAW, VARIANT 15 - ANALYZED AUTOMOTIVE CAN-BUS
=====
Результат обробки даних: RPM = ((Data[0] << 8) | Data[1]) * 0.25
| Рядок | Час, мс | CAN ID | DLC | Data_Bytes | Стан шини | RPM |
|-----|-----|-----|-----|-----|-----|-----|
| 3 | 1800 | 8C9 | 8 | 1E 20 00 00 00 00 00 00 | ACTIVE | 1528.00 |
| 4 | 1250 | 8C9 | 9 | 1F 40 00 00 00 00 00 00 | ACTIVE | 2048.00 |
| 5 | 1500 | 8C9 | 8 | 20 00 00 00 00 00 00 00 | PASSIVE | 2048.00 |
| 6 | 1750 | 8C9 | 8 | 00 00 00 00 00 00 00 00 | ACTIVE | 0.00 |
|-----|-----|-----|-----|-----|-----|-----|
ПІДСУМКОВА СТАТИСТИКА
|-----|
Рядки даних у файлі: 8
Нормалізованих рядків: 4
Коректно розібрано CAN-кадри: 4
Підтверджено рядки (format/validation): 4
Мінімальні оберти двигуна: 0.00 RPM
Максимальні оберти двигуна: 2048.00 RPM
Середні оберти двигуна: 1494.00 RPM
Кадри за станом шини: ACTIVE=3, PASSIVE=1
=====
(ce_lab_env) PS C:\Users\user\Desktop\python\python-labs\lab5>
```

Рисунок 1.1 – Скриншот вхідного журналу CAN-bus і результату його обробки

У редакторі показано журнал із коректними рядками, рядком із надлишковими пробілами та навмисно помилковими записами. У терміналі наведено повідомлення валідації, форматovanу таблицю чотирьох кадрів і агреговану статистику за станом шини.

1.3 Програмний код

1.3.1 Модуль нормалізації тексту

Файл text_sanitizer.py

```
"""Helpers for normalising raw CAN-bus log lines before regex parsing."""

from __future__ import annotations

import re

# Keep ordinary whitespace for the next step, but remove non-printing control
# characters which must not become part of a CAN frame field.
_NON_PRINTING_CONTROL_CHARACTERS = re.compile(r"[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]")
_WHITESPACE_RUN = re.compile(r"\s+")

def sanitize_log_line(raw_line: str) -> str:
    """Return a canonical one-line representation of a raw log record.
```

The function removes non-printing control characters, trims leading and trailing whitespace and converts runs of spaces or tabs into one space. Delimiters and the hexadecimal payload itself are deliberately preserved for the regular expression in :mod:`log\_parser`.

```
without_controls = _NON_PRINTING_CONTROL_CHARACTERS.sub("", raw_line)
return _WHITESPACE_RUN.sub(" ", without_controls).strip()
```

1.3.2 Модуль розбору журналу

Файл log_parser.py

```
"""Laboratory work 5, variant 15: CAN-bus log parsing and RPM decoding.
```

```
The input format is deliberately textual so that a compiled regular expression
with named capture groups can turn each log line into a structured CAN frame.
The first two data bytes encode engine speed as an unsigned big-endian value in
quarters of one RPM: ``RPM = ((byte0 << 8) | byte1) * 0.25``.
"""
```

```
from __future__ import annotations
```

```
import re
from collections import Counter
from dataclasses import dataclass
from pathlib import Path
```

```
from text_sanitizer import sanitize_log_line
```

```
CAN_FRAME_PATTERN = re.compile(
    r"^Timestamp_MS:\s*(?P<timestamp_ms>\d+)\s*\|\s*"
    r"CAN_ID:\s*(?P<can_id>[0-9A-Fa-f]{3})\s*\|\s*"
    r"DLC:\s*(?P<dlc>\d+)\s*\|\s*"
    r>Data_Bytes:\s*(?P<data_bytes>[0-9A-Fa-f]{2})(?:\s+[0-9A-Fa-f]{2})*\s*\|\s*"
    r"Bus_State:\s*(?P<bus_state>[A-Za-z_]+)$"
)
```

```
ALLOWED_BUS_STATES = frozenset({"ACTIVE", "PASSIVE", "OFF"})
RPM_SCALE = 0.25
INPUT_FILE = Path(__file__).resolve().with_name("input_logs.txt")
```

```
@dataclass(frozen=True)
```

```
class CanFrame:
    """One semantically valid CAN engine-speed frame."""
```

```
    source_line: int
    timestamp_ms: int
    can_id: str
    dlc: int
    data_bytes: tuple[int, ...]
    bus_state: str
    rpm: float
```

```
@dataclass
```

```
class ParseStatistics:
    """Counters that make parsing and normalisation results auditable."""
```

```
    data_lines: int = 0
    normalized_lines: int = 0
    rejected_lines: int = 0
```

```
def decode_engine_rpm(data_bytes: tuple[int, ...]) -> float:
    """Decode RPM from the first two bytes of a CAN engine-speed payload."""
```

```
    if len(data_bytes) < 2:
        raise ValueError("Для декодування RPM потрібні щонайменше два байти даних.")
```

```
    raw_speed = (data_bytes[0] << 8) | data_bytes[1]
    return raw_speed * RPM_SCALE
```

```
def build_frame(groups: dict[str, str], line_number: int) -> CanFrame:
    """Validate regex groups and convert them into a :class:`CanFrame`.
```

```
The variant requires a standard 11-bit CAN identifier, a DLC from 1 to 8,
exactly ``DLC`` hexadecimal bytes and a recognised bus state. RPM adds the
extra physical requirement of at least two payload bytes.
```

```

"""

timestamp_ms = int(groups["timestamp_ms"])
can_id_value = int(groups["can_id"], 16)
dlc = int(groups["dlc"])
data_bytes = tuple(int(byte, 16) for byte in groups["data_bytes"].split())
bus_state = groups["bus_state"].upper()

if can_id_value > 0x7FF:
    raise ValueError("CAN_ID має належати стандартному 11-бітному діапазону 000..7FF.")
if not 1 <= dlc <= 8:
    raise ValueError("DLC має бути в діапазоні 1..8.")
if len(data_bytes) != dlc:
    raise ValueError(
        f"DLC={dlc}, але в полі Data_Bytes передано {len(data_bytes)} байт(и).")
)
if bus_state not in ALLOWED_BUS_STATES:
    allowed = ", ".join(sorted(ALLOWED_BUS_STATES))
    raise ValueError(f"Bus_State має бути одним з: {allowed}.")

return CanFrame(
    source_line=line_number,
    timestamp_ms=timestamp_ms,
    can_id=f"{can_id_value:03X}",
    dlc=dlc,
    data_bytes=data_bytes,
    bus_state=bus_state,
    rpm=decode_engine_rpm(data_bytes),
)

def parse_can_log(file_path: Path) -> tuple[list[CanFrame], ParseStatistics]:
    """Read, normalise, parse and validate each non-comment CAN log line."""

    if not file_path.is_file():
        raise FileNotFoundError(f"Вхідний файл журналу не знайдено: {file_path}")

    frames: list[CanFrame] = []
    statistics = ParseStatistics()

    with file_path.open(encoding="utf-8") as input_file:
        for line_number, raw_line in enumerate(input_file, start=1):
            raw_without_newline = raw_line.rstrip("\r\n")
            normalized = sanitize_log_line(raw_line)

            if not normalized or normalized.startswith("#"):
                continue

            statistics.data_lines += 1
            if normalized != raw_without_newline:
                statistics.normalized_lines += 1

            match = CAN_FRAME_PATTERN.fullmatch(normalized)
            if match is None:
                statistics.rejected_lines += 1
                print(f"ПОМИЛКА РОЗБОРУ, рядок {line_number}: {normalized}")
                continue

            try:
                frames.append(build_frame(match.groupdict(), line_number))
            except ValueError as error:
                statistics.rejected_lines += 1
                print(f"ПОМИЛКА ВАЛІДАЦІЇ, рядок {line_number}: {error}")

    return frames, statistics

def format_bytes(data_bytes: tuple[int, ...]) -> str:
    """Return an uppercase hexadecimal array suitable for the report table."""

    return " ".join(f"{byte:02X}" for byte in data_bytes)

def print_report(frames: list[CanFrame], statistics: ParseStatistics) -> None:
    """Print a deterministic diagnostic table and aggregate RPM statistics."""

    separator = "=" * 119
    row_separator = "-" * 119
    print("\n" + separator)
    print(f"'ЛБ5, ВАРІАНТ 15 – АНАЛІЗ АВТОМОБІЛЬНОЇ ШИНИ CAN-BUS':^119}")
    print(separator)
    print("Формула декодування датчика обертів: RPM = ((Data[0] << 8) | Data[1]) × 0.25")
    print(
        f"| {'Рядок':^6} | {'Час, мс':^10} | {'CAN ID':^8} | {'DLC':^5} | "
        f"{'Data_Bytes':^23} | {'Стан шини':^11} | {'RPM':^12} |"
    )

```

```

    )
    print(row_separator)

    for frame in frames:
        print(
            f"| {frame.source_line:^6d} | {frame.timestamp_ms:^10d} | {frame.can_id:^8} | "
            f"{frame.dlc:^5d} | {format_bytes(frame.data_bytes):<23} | "
            f"{frame.bus_state:^11} | {frame.rpm:>10.2f} | "
        )

    print(separator)
    print("ПІДСУМКОВА СТАТИСТИКА")
    print(row_separator)
    print(f"Рядків даних у файлі: {statistics.data_lines}")
    print(f"Нормалізовано рядків: {statistics.normalized_lines}")
    print(f"Коректно розібрано CAN-кадрів: {len(frames)}")
    print(f"Відхилено рядків (format/validation): {statistics.rejected_lines}")

    if frames:
        rpm_values = [frame.rpm for frame in frames]
        states = Counter(frame.bus_state for frame in frames)
        print(f"Мінімальні оберти двигуна: {min(rpm_values):.2f} RPM")
        print(f"Максимальні оберти двигуна: {max(rpm_values):.2f} RPM")
        print(f"Середні оберти двигуна: {sum(rpm_values) / len(rpm_values):.2f} RPM")

        print(
            "Кадри за станом шини: "
            + ", ".join(f"{state}={states[state]}" for state in sorted(states))
        )
    else:
        print("RPM-статистика недоступна: немає коректних кадрів.")
    print(separator)

def main() -> None:
    """Run the complete CAN-bus parsing pipeline for the supplied input file."""

    frames, statistics = parse_can_log(INPUT_FILE)
    print_report(frames, statistics)

if __name__ == "__main__":
    main()

```

1.4 Відповіді на контрольні питання

1.4.1 Внутрішня модель рядків CPython

PEP 393 застосовує гнучке внутрішнє подання Unicode: рядок зберігає символи у 1-, 2- або 4-байтовому представленні залежно від найбільшого кодового пункту. Для ASCII достатньо одного байта на символ. Поява символу кирилиці потребує щонайменше двобайтового представлення всього рядка, оскільки він зберігається в одному однорідному форматі.

1.4.2 Конкатенація та str.join()

Рядки є незмінними, тому += у циклі створює новий рядок на кожній ітерації і повторно копіює вже накопичені символи. Сумарна вартість наближається до $O(N^2)$. Метод str.join() попередньо визначає потрібний розмір і формує результат за один прохід, тому має лінійну складність $O(N)$.

1.4.3 re.match(), re.search() і re.finditer()

re.match() перевіряє лише початок рядка, re.search() шукає перший збіг у будь-якій позиції, а re.finditer() повертає ітератор усіх об'єктів Match. finditer доцільний для великих журналів або коли потрібні позиції збігів та іменовані групи без створення великого проміжного списку, на відміну від findall().

1.4.4 Катастрофічне повернення

Катастрофічне повернення виникає, коли вкладені або неоднозначні квантифікатори утворюють велику кількість можливих шляхів зіставлення. Для невдалого рядка рушій може витратити експоненційний час. Його уникають точними класами символів, якорями, обмеженими квантифікаторами та не жадібними квантифікаторами там, де потрібна найкоротша відповідність.

1.4.5 Іменовані групи захоплення

Конструкція (?P<name>pattern) зіставляє pattern і зберігає фрагмент під назвою name. Після успішного зіставлення значення доступні як `match.group("name")`, а весь набір полів отримується словником через `match.groupdict()`. Саме цей словник у програмі передається до `build_frame()`.

1.4.6 Переваги f-рядків

f-рядки дозволяють розміщувати вирази Python безпосередньо в літералі та задавати вирівнювання, ширину і точність у міні-мові форматування. Вони читаються простіше за % і `str.format()`, а компілятор використовує спеціалізовані байт-кодові операції `FORMAT_VALUE` та `BUILD_STRING`, без окремого синтаксичного розбору шаблону під час кожного виклику `format()`.

1.5 Репозиторій та висновки

Вихідний код лабораторної роботи розміщено у репозиторії: <https://github.com/sobof1/python-labs>, каталог lb5.

У роботі створено аналізатор текстового журналу CAN-bus для варіанта 15. Нормалізація рядків, скомпільований регулярний вираз та семантична валідація відокремлюють коректні кадри від помилкових записів. Іменовані групи дають зрозумілий перехід від тексту до структури `CanFrame`, а f-рядки забезпечують читабельне вирівняне подання даних і підсумкової статистики RPM.